QuroGrid 화면 정의(TOON) 설계서 v0.2

- 문서 상태: v0.2 (2026-10-29) — 6.1절 C-01~C-05·T-11 확정 (Addendum A-44, QuroHarness ADR-0058). 6.2절 나머지 T-xx는 잠정안
- 대상: `Quro.Grid.Definition` (파서·검증기·불변 모델·식 평가기), 그리고 그 모델을 `Quro.Grid` / `Quro.Grid.Data`에 연결하는 매핑
- 입력 문서: QuroERP TO BE 설계서 v0.1「Procentro 화면·Grid XML 정의를 TOON 메타데이터로 전환하는 TO BE 설계 로직」(2026-10-29, 이하 **TOBE**. 원본 PDF와 `sd.so.entry.screen.toon` 예제는 QuroHarness `analysis/04-tech-architecture/tobe-ui-grid-toon/`에 보관, 비공개), `tests/scenarios/GS-D.md` (v0.1)
- 우선순위: Spec Addendum > Spec v1 > **이 문서** > TOBE. TOBE와 QuroGrid 확정 결정(D-01~D-03)이 다르면 QuroGrid 결정을 따르고 6장에 충돌로 적는다
- Harness(테스트 시나리오): `tests/scenarios/GS-D.md` v0.2

  참고: CLAUDE.md·`.claude/rules/screen-definition-toon.md`가 가리키는 `docs/QuroGrid_TOON_ScreenDef_v0.1.pdf`는 저장소에 없다. v0.2는 v0.1을 이어 고치는 대신 TOBE 스키마를 기준으로 다시 썼고, v0.1의 개념(`cond`, `refs`, `summary`, `queries`, `map`, `edit: new`)은 7.4절에서 TOBE 스키마로 옮겼다.

---

1. 범위와 책임 경계

TOBE는 QuroERP 전체(화면 메타 → Blazor 화면 → API → Dapper → PostgreSQL)를 다룬다. 이 저장소는 그중 **UI 메타의 해석**과 **그리드**만 맡는다. QuroGrid는 QuroERP를 모른다 (CLAUDE.md 4장).

TOBE 구성요소	맡는 곳	이 저장소의 산출물
TOON 서브셋 파서 (D3)	**QuroGrid** `Quro.Grid.Definition`	`ToonReader` (strict)
`ScreenDef` 불변 모델 (TOBE 3.2)	**QuroGrid** `Quro.Grid.Definition`	`ScreenDef`, `FormDef`, `FieldDef`, `GridDef`, `ColumnDef`, `RuleDef`, `ValidateDef`
의미 검증 (H1·H3·H4·H7 일부)	**QuroGrid** `Quro.Grid.Definition`	`ScreenDefValidator` + 주입 카탈로그 `IQuroDefinitionCatalog`
`FieldModeResolver` (TOBE 2.5)	**QuroGrid** `Quro.Grid.Definition`	순수 함수. 폼·그리드가 같이 쓴다
규칙 식(`expr`) 평가 (P4·P5)	**QuroGrid** `Quro.Grid.Definition`	`RuleExpression` — 클라이언트(즉시 표시)와 서버(권위 재계산)가 같은 코드 사용
그리드 렌더링	**QuroGrid** `Quro.Grid`	`<QuroGrid Definition="@gridDef">` → `QuroColumn` 매핑 (T-11)
DataSet 스키마	**QuroGrid** `Quro.Grid.Data`	`GridDef` → 행 키·`FieldRule` 등록
`UiMetaRegistry` (시작 시 전체 로드·캐시)	QuroERP	— (`ScreenDefLoader`만 제공)
`QuroForm` / `QuroField` 렌더러, Toolbar	QuroERP	— (`FieldModeResolver`만 제공)
`code:` / `lookup:` 해석, `ILookupProvider`, 공통코드 API	QuroERP	— (참조 문법만 파싱·검증)
`@today`, `@company.*`, `@setting:*` 기본값 해석	QuroERP	— (토큰 문법만 파싱·검증)
권한(`perm`, `readOnlyFor`) 판정, `lockWhen` 평가 대상 값	QuroERP	`lockWhen` 식 파싱·평가 함수

TOBE 구성요소	맡는 곳	이 저장소의 산출물
i18n 리소스 TOON (`i18n/ko/sd.toon`)	QuroERP	같은 `ToonReader` 로 읽을 수 있음. 키 목록은 카탈로그로 주입
`lookup.toon`	QuroERP	같은 `ToonReader` . 스키마는 6장 T-08
화면 정의 파일 `*.screen.toon`	QuroERP	테스트용 골든 파일만 `tests/Quro.Grid.Definition.Tests/Golden/`
저장 API, Pricing, 채번, Dapper (R1·R2·R6~R9)	QuroERP	—
`xml2toon` 변환기·`toonlint` ·골든 (H5)	QuroHarness `harness/` (I-07, 비공개 — 레거시 자산이 공개 저장소로 나가지 않음)	—

의존 방향은 그대로다: `Quro.Grid` → `Quro.Grid.Definition` → `Quro.Grid.Data` . `Quro.Grid.Definition` 은 Blazor를 참조하지 않으므로 **QuroERP 서버(API)도 같은 패키지를 참조해** 같은 메타로 검증·재계산한다 (P5).

---

2. 설계 원칙

TOBE 2.1의 P1~P7을 그대로 받는다. QuroGrid 쪽에서 원칙을 지키는 방법:

#	원칙	QuroGrid 쪽 구현
P1	One Screen, One File	파서는 파일 1개 = `ScreenDef` 1개. `lookup.toon` 은 별도 `LookupDef`
P2	Name, never Position	모든 참조는 id. `ColumnDef` · `FieldDef` 에 인덱스 API 없음. 행 값은 `QuroDataSet.GetValue(rowKey, field)`
P3	No SQL in UI	검증기가 SQL 문장 패턴을 오류로 막음 (V-30). `source` · `lookup` 은 불투명 참조
P4	Declare UI, Compute on Server	`rules` 의 식을 `RuleExpression` 으로 평가. 그리드는 표시용으로만 쓰고, 서버가 같은 식으로 다시 계산
P5	Same metadata, both sides	`Quro.Grid.Definition` 은 UI 무관 → 서버도 참조. `req` · `max` · `validate` 를 양쪽이 같은 `ValidateDef` 로 검사
P6	i18n by default	`label` · `msg` · `title` 은 리소스 키만. 문구가 들어오면 오류 (V-21)
P7	Harness-verifiable	strict 파서(`[N]` ·헤더 개수), 열거값, 유일 id. 시나리오는 GS-D.md

---

3. TOON 서브셋 문법 (파서 계약)

TOON 공식 스펙은 Working Draft다. 정의 파일에 실제로 쓰는 부분만 지원한다 (T-06). 지원하지 않는 구문은 **로드 실패**다 (조용히 무시하지 않는다).

구문	예	규칙
스칼라	`id: sd.so.entry`	`키: 값` . 키는 `[A-Za-z][A-Za-z0-9]*` (하이픈·점 금지)
중첩 객체	`screen:` 다음 줄부터 들여쓰기 +2	들여쓰기는 공백 2칸 단위. 탭 금지
원시값 배열	`toolbar[9]: query,new,save,...`	개수 = `[N]` . 다르면 오류

구문	예	규칙
표(균일 객체 배열)	`columns[10]{id,label,...}:` + 값 행	행 수 = `[N]`, 행의 값 수 = 헤더 필드 수. 행은 헤더보다 2칸 더 들여씀
목록(비균일 객체 배열)	`grids[1]:` + `- id: lines`	항목 수 = `[N]`. `-` 다음 키부터 같은 객체
주석	`# AS IS : so.xml`	**줄 맨 앞**(들여쓰기 뒤 포함)의 `#`만. 값 뒤 `#`는 값의 일부 (D3 잠정)
구분자	`,`	고정. 탭·`|` 구분자 헤더는 지원 안 함
따옴표	`"lookup:salesOrder"`, `""`	빈 문자열, `,` `:` `"` `#` 앞뒤 공백, `true`/`false`/`null`·숫자처럼 보이는 문자열은 `"…"` 필수. 이스케이프는 `\"` `\\` 만
셀 안 목록	`"itemName|unitPrice|stdPrice"`	`|`로 나눈 id 목록 (TOON 문법이 아니라 이 스키마의 규약)
타입 값	`true` `false` `150` `0.5`	따옴표 없으면 불·수. `"true"`는 문자열
빈 값	`""` 또는 비움	표 셀에서 `""`와 빈 칸은 같은 `null`로 읽음

strict 오류는 모두 `파일명:줄` **형식으로 보고**하고 첫 오류에서 멈추지 않는다 (한 번에 모아 보고). 정의 파일을 범용 TOON 인코더로 다시 저장하지 않는다 (주석·정렬 소실).

---

4. 스키마 — `*.screen.toon` v0.2

TOBE 2.4·2.7을 기준으로 하고, QuroGrid 그리드 기능에 맞춰 열 몇 개를 더했다 (★ 표시, 모두 선택 열이며 표 헤더에 없으면 기본값).

4.1 `screen`

키	필수	형식	설명
`id`	예	`{module}.{doc}.{screen}` 소문자·점	파일명과 같아야 함 (`sd.so.entry.screen.toon`)
`module`	예	대문자	`SD`, `MM` … (값은 불투명, QuroGrid는 해석 안 함)
`title`	예	리소스 키	
`version`	예	정수 ≥ 1	메타 스키마가 아니라 **이 화면 정의의** 버전
`api`	예	`/`로 시작하는 경로	QuroGrid는 해석 안 함. 저장 URL로 앱에 넘겨줌
`key`	예	form 필드 id	문서 키 (`soNo`)
`layout`	예	`form-grid` | `form` | `grid`	
`perm.read` / `perm.write`	예	권한 id (불투명)	
`lockWhen`	아니오	5.6절 식	참이면 FormState = Locked

4.2 `toolbar[N]`

명령 id 목록. 허용값: `query`, `new`, `save`, `delete`, `addRow`, `deleteRow`, `undoRow`, `uploadCsv`, `exportXlsx` ★, `exportCsv` ★, `print`. `addRow`·`deleteRow`·`undoRow`·`uploadCsv`·`export*`는 `grids`가 있어야 한다 (V-14). 그리드 명령은 QuroGrid API로 매핑된다 (7.3절).

4.3 `form`

- `cols` : 정수 1~4
- `fields[N]{id,label,type,mode,max,case,default,source,span,on}`

열	형식	설명
`id`	camelCase	화면 안 유일 (폼 필드와 그리드 id가 겹치면 안 됨)
`label`	리소스 키	`hidden` 이면 `""` 허용
`type`	5.1절	
`mode`	5.2절	
`max`	정수 또는 `""`	문자열 최대 길이. `text` · `textarea` · `password` 에만
`case`	`upper` | `lower` | `""`	입력 변환
`default`	5.5절 토큰 또는 리터럴	
`source`	5.4절 참조	`select` · `radio` 는 필수
`span`	정수 0~ `cols`	0은 `hidden` 에만
`on`	핸들러 id 또는 `""`	변경 시 앱 핸들러. QuroGrid는 이름만 전달

- `readOnlyFor[N]{perm,fields}` : 권한 id + `|` 필드 목록. 해당 권한 사용자는 그 필드가 읽기 전용 (FieldModeResolver의 `permReadOnly`)

4.4 `grids[N]` (목록)

키	필수	설명	QuroGrid 연결
`id`	예	화면 안 유일	`<QuroGrid>` 식별
`bind`	예	저장 DTO 컬렉션 이름	H2 계약 테스트 키 (QuroERP)
`rowKey`	예	서버 행 키 필드	열에 없어도 됨 (숨은 키). DataSet 업무 키 (7.2)
`initRows`	아니오 (0)	처음 빈 행 수	`AddBlankRow` × initRows, 그리기 1회 (A-46)
`clipboard`	아니오 (true)	복사·붙여넣기 허용	`Clipboard` (G-03 ✅)
`pageSize` ★	아니오 (0)	0 = 메모리 모드, >0 = 서버 페이징 (A-24)	`ItemsProvider` 사용 여부
`frozen` ★	아니오 (0)	왼쪽 고정 열 수	`FrozenColumns`
`deletedRows` ★	아니오 (`hide`)	`hide` | `strike` (O-02)	`DeletedRows`

`columns[N]{id,label,type,width,edit,req,fmt,lookup,sum,hidden}`

선택 열★: `align`, `min`, `max`, `source`, `currency`, `filter`, `sort`, `group`

열	형식	기본	설명
`id`	camelCase	—	DataSet 필드명과 같음 (P2)
`label`	리소스 키	—	`HeaderKey`
`type`	5.3절	—	
`width`	정수 40~800	120	px
`edit`	`true` | `false` | `new`	—	`new` = 추가 행에서만 편집 (v0.1 `edit: new`, TOBE의 newOnly). G-01
`req`	불	false	필수 (`edit=false`이면 true일 수 없음, V-12)
`fmt`	.NET 서식 또는 `""`	`""`	`#,##0.###`, `N2` … 표시용. 저장값은 언어 무관
`lookup`	lookup id 또는 `""`	`""`	코드도움 (`type=text`일 때만) → Popup 셀
`sum`	불	false	합계 행·소계 (`number`·`money`만)
`hidden`	불	false	화면에 안 보임, DataSet에는 있음
`align` ★	`left` | `center` | `right`	타입별	숫자는 right
`min` ★/`max` ★	수	—	`FieldRule` 범위 (포함)
`source` ★	`code:XXX`	—	`type=select`면 필수
`currency` ★	행 필드 id	—	`type=money`일 때 통화 필드 → Currency 셀
`filter` ★·`sort` ★	불	true	헤더 필터·정렬 허용 (G-04)
`group` ★	불	false	그룹핑 허용 (A-42)

`rules[N]{on,action,target,expr}`

열	형식	설명
`on`	`{columnId}.change` | `{gridId}.change`	트리거. `{gridId}.change` = 행 추가·삭제·값 변경
`action`	`calc` | `sum` | `lookup` | `set`	
`target`	열 id, 폼 필드 id(`sum`), 또는 | 목록(`lookup`)	
`expr`	5.7절 식	`lookup`이면 `{lookupId}.{query}({인수})`

- `calc`: 같은 행의 `target` 열 = 식 결과. 대상 열은 `edit=false`여야 함 (V-15)
- `sum`: 폼 필드 `target` = 그리드 집계. 대상 필드는 `mode=ro` (V-16, v0.1 `summary.target` 규칙)
- `lookup`: 앱이 조회 후 `target` 열들에 값 채움. QuroGrid는 트리거·대상만 전달 (데이터 조회 안 함)
- `set`: `target` = 식 결과 (상수 포함)

- 순환 금지: `calc` 대상이 다른 `calc` 의 입력이고 그 반대도 성립하면 오류 (V-17)

`validate[N]{target,rule,msg}`

`rule` 문법 (5.8절): `gt 0`, `ge 0`, `lt n`, `le n`, `ne 0`, `requires {formFieldId}`, `stock …` (앱 위임). `msg` 는 리소스 키.

---

5. 열거값·미니 문법

5.1 form `type`

`text`, `textarea`, `password`, `select`, `radio`, `checkbox`, `date`, `number`, `money`, `money4`, `hidden`

5.2 `mode` (TOBE 2.5 매트릭스)

mode	AS IS tag	New	Loaded	Locked / 읽기 권한
`opt`	x1	편집	편집	읽기
`req`	x2	편집·필수	편집·필수	읽기
`key`	x3	편집·필수	읽기	읽기
`newOnly`	x5	편집	읽기	읽기
`ro`	x4	읽기	읽기	읽기
`auto`	(신규)	읽기 (채번 예정)	읽기	읽기
`hidden`	type=hidden	숨김	숨김	숨김

`FieldModeResolver.Resolve(mode, state, permReadOnly)` → `FieldState(Visible, Editable, Required)`. TOBE 2.5 코드를 그대로 쓴다. 그리드 열의 `edit` · `req` 도 같은 리졸버로 환산한다: `edit=true,req=true` → `req`, `edit=true` → `opt`, `edit=new` → `newOnly` (행 상태 Inserted = New), `edit=false` → `ro`.

5.3 column `type`

`text`, `number`, `money`, `date`, `select`, `checkbox`, `button` ★. → QuroGrid `CellType` 매핑은 7.1절.

5.4 데이터소스 참조

`code:{MAJOR}` (공통코드, 예 `code:S200`), `lookup:{id}` (엔터티 조회·팝업, 예 `lookup:customer`). 열 `lookup` 칸은 접두어 없이 `item` 처럼 id만 쓴다 (TOBE 예제와 같음). 존재 여부는 카탈로그로 검사 (V-22).

5.5 기본값 토큰

`@today`, `@now`, `@company.{key}`, `@setting:{key}`, `@user.{key}`. 해석은 앱(QuroERP). QuroGrid는 문법만 검사한다. `@`로 시작하지 않으면 리터럴.

5.6 `lockWhen` 식

```
cond   := field "in" "[" value ("," value)* "]"
        | field ("=" | "!=") value
        | cond ("and" | "or") cond
```

예: `status in [RE,CL]`. 필드는 폼 필드 id여야 함 (V-18). `LockExpression.Evaluate(Func<string, object?> valueOf)` 제공.

5.7 규칙 식 (`expr`) — 화이트리스트

```
expr   := term (("+" | "-") term)*
term   := factor (("*" | "/") factor)*
factor := number | id | "(" expr ")" | "-" factor | func "(" args ")"
func   := sum | min | max | round | abs | coalesce
```

- `id`는 같은 그리드의 열 id, 또는 `sum`의 인수로 열 id. 폼 필드는 `lookup` 인수에서만
- 계산은 `decimal`. `null`·빈 값은 0 (TOBE R11). 0으로 나누기 = 결과 `null` + 경고 (예외 없음). `null` 결과는 0으로 나누기에서만 나오므로 호출자는 null을 경고로 표시한다
- 함수: `sum(열)` 열 전체 합(빈 값 제외), `min`·`max`(인수 1개 이상), `round(x[, 자릿수 0~28])` 반올림은 0에서 먼 쪽(MidpointRounding.AwayFromZero), `abs(x)`, `coalesce(a, b, …)` 비어 있지 않은 첫 값
- 반올림: 결과를 `target` 열의 `fmt` 소수 자릿수로 반올림하지 않는다 — 저장값 반올림은 서버 Pricing 규약 (T-13)
- 문자열 연결·비교·사용자 함수·리플렉션 없음 → 식으로 임의 코드를 실행할 수 없다 (H7)

5.8 `validate.rule`

rule	의미	실행
`gt n` / `ge n` / `lt n` / `le n` / `ne n`	수 비교	QuroGrid (`FieldRule` 확장, G-02) + 서버
`requires {fieldId}`	폼 필드가 비어 있으면 이 열 입력 불가	앱 (`OnCellValueChanging` 취소) + 서버
`stock …` 등 그 밖	업무 판정	앱. QuroGrid는 `SetCellMark`로 표시만 (Spec 4.9)

---

6. 결정 필요 — 스테이블영 확인

Critical(아키텍처·공개 API·데이터 모델) 항목은 확정 전까지 잠정안으로 문서·테스트를 쓰되 구현하지 않는다. 6.1절은 2026-10-29 스테이블영이 제안대로 확정했다 (Addendum A-44).

6.1 TOBE와 QuroGrid 결정의 충돌

ID	충돌	TOBE	QuroGrid	결정 (2026-10-29)
C-01	그리드 컴포넌트 (TOBE D1)	MudBlazor 또는 Radzen 그리드 권고	D-01 자체 제작, MudDataGrid 금지. 0.1.0-alpha.1 배포 완료	**확정: QuroGrid 유지**. TOBE D1은 "QuroGrid로 확정" (QuroHarness ADR-0058)
C-02	파서·모델 위치	QuroERP `src/Quro.Ui.Meta/`	`Quro.Grid.Definition` 패키지 (I-02 대기)	**확정: `Quro.Grid.Definition` 이 파서·모델·검증·리졸버·식 평가 전부**. QuroERP는 레지스트리·폼 렌더러·소스 해석만. 서버도 이 패키지 참조 (P5). I-02 해소
C-03	저장 payload	`header + lines[{rowState: Added/Modified/Deleted}] + rowVersion`	`ChangeSet {inserted[], updated[], deleted[]}` + 행 `version` (Spec 4.4)	**확정: QuroDataSet 형식 유지**. QuroERP가 커맨드 DTO로 옮긴다 (QuroHarness ADR-0058에 기록). 필요하면 `ChangeSet.ToRowStateList()` 보조 함수 추가 (비파괴)
C-04	셀 수식	`rules.calc` 식을 메타에 선언	grid-features: "셀 수식은 v1 범위 밖, 금액 계산은 앱이 `OnCellValueChanged` 에서"	**확정: 그리드는 식을 모른다.** `RuleExpression` 은 Definition에 두고, 연결은 앱 또는 `GridDef` 연결 도우미가 `OnCellValueChanged` 에서 한다. 엑셀식 셀 수식이 아니라 열 규칙이므로 "셀 수식은 v1 범위 밖"과 충돌하지 않는다
C-05	레거시 변환기	`tools/xml2toon`	I-07: QuroERP 쪽 도구	**확정: QuroGrid에서 빼서 QuroHarness `harness/` 로 이동** (QuroHarness P-127 4번 추천과 같음, 레거시 자산 비공개). QuroGrid의 린트는 C# `ScreenDefValidator` + CLI(T-12)
T-11	그리드 연결 API	—	consumer-contract 4장 `[미정]`	**확정: `<QuroGrid Definition="@gridDef">` 파라미터** (공개 API, 묶음 D-5에서 GS 먼저)

6.2 화면 정의 미결 항목 (T-xx 재정리)

ID	내용	잠정안
T-01	스키마 기준	TOBE 2.4 + 4장 ★ 확장 열
T-02	주석 `#` (TOBE D3)	줄 맨 앞만 허용
T-03	리소스(i18n) 형식 (TOBE D6)	TOON `labels[N]{key,text}`. 키 목록은 카탈로그로 주입하므로 형식이 바뀌어도 QuroGrid 영향 없음
T-04	화면 파일 단위 (TOBE D7)	1화면 1파일, 공통 팝업은 `lookup.toon`
T-05	그리드 `edit` 값	`true` / `false` / `new`
T-06	TOON 파서	자체 서브셋 파서(새 의존성 없음) + 공식 스펙 픽스처 중 서브셋 해당분으로 검증
T-07	로드 시점	앱 시작 시 전체 로드·검증, 실패 시 앱 시작 실패 (개발 환경은 핫리로드)
T-08	`lookup.toon` 스키마	v0.3로 미룸. v0.2는 lookup id 존재만 검사
T-09	`select` 값	값=코드, 표시=언어별 명칭 (TOBE 3.5). 이름 저장 금지
T-10	할인 식 부호 (TOBE D4·R3)	업무 결정 사항. QuroGrid는 식을 그대로 평가할 뿐 — 골든 파일은 차감식
T-12	검증 CLI	**결정 (A-46)**: `dotnet quro-deflint <폴더>` dotnet tool
T-13	계산 결과 반올림	그리드 표시는 `fmt`, 저장값은 서버 규약

7. QuroGrid 매핑

7.1 열 → `QuroColumn`

TOON 열	`QuroColumn`	비고
`id`	`Field`	
`label`	`HeaderKey`	
`type=text`	`Type=Text`	`lookup` 이 있으면 `Type=Popup` + `Lookup` = 앱의 `LookupFor(lookupId, 입력)` (A-46)
`type=number`	`Type=Number`	
`type=money`	`Number` (`currency` 없음) / `Currency` (`currency` 있음 → `CurrencyField`)	
`type=date`	`Type=Date`	
`type=select`	`Type=Combo`, `Options` = 앱의 `OptionsFor(source)` (A-46)	
`type=checkbox`	`Type=Check`	
`type=button`	`Type=Button`, `TextKey` = `label`	
`width`	`Width`	
`edit=false`	`ReadOnly=true`	
`edit=new`	`NewRowOnly=true`	G-01 ✅
`req`	`Required`	
`fmt`	`Format`	
`sum`	`Aggregate=Sum`	
`hidden=true`	열을 만들지 않음 (DataSet 필드만)	
`min` / `max`	`Min` / `Max`	
`filter` / `sort` / `group`	`Filterable` / `Sortable` / `Groupable`	G-04 ✅

7.2 그리드 → `QuroGrid` · `QuroDataSet`

- `rowKey`: QuroDataSet의 내부 행 키(int)와 별개인 **업무 키**. 저장 payload에 그대로 실림. 추가 행은 null (서버 채번, R6)
- `initRows` → `AddBlankRow` × initRows, 그리기 1회 (R3). 손대지 않은 빈 행은 저장·검증·이탈 경고 대상이 아니다 (A-46)
- `pageSize>0` → `ItemsProvider`; `rules` 의 `sum` 은 서버 `Totals` 로 (A-24)
- `validate gt/ge/lt/le/ne` → `DataSet.SetRule` (G-02 ✅): `MinExclusive`·`MaxExclusive`·`NotEqual`, 메시지 = `msg` (`FieldRule.MessageKey`)

7.3 툴바 → 그리드 API

`addRow` → `AddRow()`, `deleteRow` → `DeleteSelectedRows()`, `undoRow` → `RevertRow(active)` (O-01), `exportXlsx` / `exportCsv` → `GetExportRequest(format)`, `uploadCsv` → 앱이 파일을 읽어 `ImportRows(fields, rows)` (G-05, A-47: 붙여넣기 경로로 끝에 행 추가). `query` · `new` · `save` · `delete` · `print` 는 앱.

7.4 v0.1 개념 → v0.2

v0.1	v0.2
`cond` (조회 조건 영역)	`form.fields` 의 조회 조건 — TOBE는 영역을 레이아웃 `area` 로 흡수. v0.2에서는 별도 섹션 없음
`refs` (참조 팝업 + `map`)	`rules` `action=lookup`, `target` = | 목록
`summary.target`	`rules` `action=sum` (V-16)
`queries.*` / `queryId`	`screen.api` + `lookup:` / `code:` 참조
`edit: new`	`edit=new` (G-01)

7.5 그리드 기능 갭 (구현 전 GS 시나리오 필요)

ID	갭	필요 공개 API (제안)
G-01	추가 행에서만 편집되는 열	✅ `QuroColumn.NewRowOnly` (A-45)
G-02	초과·미만(배타) 범위, `ne`	✅ `FieldRule.MinExclusive` · `MaxExclusive` · `NotEqual` (+ `MessageKey`) (A-45)
G-03	복사·붙여넣기 끄기	✅ `QuroGrid.Clipboard` (A-45)
G-04	열별 필터·정렬·그룹 허용	✅ `QuroColumn.Filterable` · `Sortable` · `Groupable` (A-45)
G-05	CSV 업로드	✅ `QuroGrid.ImportRows(fields, rows)` → `ImportResult` (A-47). 파싱은 앱

---

8. 의미 검증 (`ScreenDefValidator`)

카탈로그는 앱이 주입한다: `IQuroDefinitionCatalog { ResourceKeys, CodeMajors, LookupIds, Handlers, Permissions }`. 오류는 `DefinitionError(Code, File, Line, Message)`.

코드	검사	Harness
V-01	`[N]` ≠ 실제 개수, 행 값 수 ≠ 헤더 수	H1
V-02	알 수 없는 키·섹션 (오타)	H1
V-03	열거값 (`type`, `mode`, `case`, `edit`, `action`, `layout`, `toolbar`)	H1
V-04	id 중복 (폼 필드·그리드·열 전체)	H1
V-05	`screen.id` ≠ 파일명	H1

코드	검사	Harness
V-10	`screen.key` 가 폼 필드에 없음	H1
V-11	`max` 가 문자열 타입 아닌 필드에	H1
V-12	`req=true` 인데 `edit=false`	H6
V-13	`sum=true` 인데 숫자 타입 아님	H1
V-14	그리드 명령인데 `grids` 없음	H1
V-15	`calc` 대상이 `edit≠false`, 또는 대상·입력 열 없음	H2
V-16	`sum` 대상이 `mode=ro` 폼 필드 아님	H2
V-17	`calc` 순환	H2
V-18	`lockWhen`·`requires` 필드 없음, 식 문법 오류	H1
V-19	`select` 인데 `source` 없음 / `lookup` 이 `text` 아닌 열에	H4
V-21	`label`·`msg`·`title` 이 리소스 키 형식 아님 또는 카탈로그에 없음	H3
V-22	`code:`·`lookup:`·열 `lookup` 이 카탈로그에 없음	H4
V-23	`on` 핸들러가 카탈로그에 없음	H4
V-24	`perm`·`readOnlyFor.perm` 이 카탈로그에 없음	H4
V-30	SQL 문장 패턴 (`select … from`, `insert into`, `update … set`, `delete from`)	H7
V-31	Clean-room 금지 식별자 (`.claude/rules/clean-room.md` 목록), snake_case id (레거시 열 이름 형식 — id는 camelCase만)	Clean-room
V-32	색 값(`#rrggbb`)으로 의미 표현 — 편집·필수 표시는 `edit`·`req` 에서 계산	Clean-room

경고(로드는 성공): 사용하지 않는 `lookup`, `hidden` 열인데 `label` 있음, `width` 가 범위 밖이라 보정됨.

---

9. 불변 모델 (공개 API)

C-02·T-11 확정(A-44)으로 승인. ✅ = 구현됨 (묶음 D-1, GS-D01·GS-D02), 나머지는 해당 묶음에서 GS 먼저.

```
namespace Quro.Grid.Definition;

// ✅ D-1 — 모델. 선택 열(align, min, max, source, currency, filter, sort, group)은 ColumnDef의 init 속성
public sealed record ScreenDef(string Id, string Module, string Title, int Version, string Api, string Key,
    ScreenLayout Layout, PermDef Perm, string? LockWhen, IReadOnlyList<ToolbarCommand> Toolbar,
    FormDef? Form, IReadOnlyList<GridDef> Grids, SourceInfo Source);
public sealed record PermDef(string Read, string Write);
public sealed record FormDef(int Cols, IReadOnlyList<FieldDef> Fields, IReadOnlyList<ReadOnlyRule> ReadOnlyFor);
```

```
public sealed record FieldDef(string Id, string? Label, FieldType Type, FieldMode Mode, int? Max, CaseRule Case,
    string? Default, string? DataSource, int Span, string? On, SourceInfo Source);
public sealed record ReadOnlyRule(string Perm, IReadOnlyList<string> Fields, SourceInfo Source);
public sealed record GridDef(string Id, string Bind, string RowKey, int InitRows, bool Clipboard, int PageSize,
    int Frozen, DeletedRowsMode DeletedRows, IReadOnlyList<ColumnDef> Columns, IReadOnlyList<RuleDef> Rules,
    IReadOnlyList<ValidateDef> Validate, SourceInfo Source);
public sealed record ColumnDef(string Id, string Label, ColumnType Type, int Width, ColumnEdit Edit, bool Req,
    string? Fmt, string? Lookup, bool Sum, bool Hidden, SourceInfo Source) { /* 선택 열 init 속성 */ }
public sealed record RuleDef(string On, RuleAction Action, IReadOnlyList<string> Targets, string Expr, SourceInfo Source);
public sealed record ValidateDef(string Target, string Rule, string Msg, SourceInfo Source);
public enum ScreenLayout { FormGrid, Form, Grid }
public enum ToolbarCommand { Query, New, Save, Delete, AddRow, DeleteRow, UndoRow, UploadCsv, ExportXlsx, ExportCsv, Print }
public enum FieldType { Text, Textarea, Password, Select, Radio, Checkbox, Date, Number, Money, Money4, Hidden }
public enum FieldMode { Opt, Req, Key, NewOnly, Ro, Auto, Hidden }
public enum CaseRule { None, Upper, Lower }
public enum ColumnType { Text, Number, Money, Date, Select, Checkbox, Button }
public enum ColumnEdit { ReadOnly, Editable, NewRowOnly }     // TOON edit: false / true / new
public enum ColumnAlign { Left, Center, Right }
public enum RuleAction { Calc, Sum, Lookup, Set }
public enum DeletedRowsMode { Hide, Strike }

// ✅ D-1 — 로드·오류
public sealed record SourceInfo(string File, int Line);
public sealed record DefinitionError(string Code, string File, int Line, string Message);   // ToString() = "file:line: code message"
public sealed class DefinitionResult<T> { public T? Value { get; } public IReadOnlyList<DefinitionError> Errors { get; } }
public static class ScreenDefLoader
{
    public static DefinitionResult<ScreenDef> Load(string path, string text);   // strict 파싱(V-01) + 키(V-02) + 값·열거값(V-03) + 구조(V-04~V-18, D-2)
}

// ✅ D-2 — 구조 검사 V-04~V-18은 Load 안에서 (V-01~V-03이 없을 때만 — 연쇄 오류 방지). 모드 리졸버
public enum FormState { New, Loaded, Locked }
public readonly record struct FieldState(bool Visible, bool Editable, bool Required);
public static class FieldModeResolver { public static FieldState Resolve(FieldMode mode, FormState state, bool permReadOnly); }
// 열 edit·req → mode 환산(5.2)은 internal ModeOf. D-5 그리드 연결에서 Quro.Grid가 쓴다 (InternalsVisibleTo)
// lockWhen 필드 존재(V-18)는 D-4 LockExpression과 함께

// ✅ D-3 — 카탈로그 검사 V-19~V-32 (V-30~V-32는 카탈로그와 무관하지만 같은 검사기에서). 기본값 토큰 문법(5.5)은 Load의 V-03
```

```
public interface IQuroDefinitionCatalog
{
    IReadOnlySet<string> ResourceKeys { get; }  IReadOnlySet<string> CodeMajors { get; }  IReadOnlySet<string> LookupIds { get; }
    IReadOnlySet<string> Handlers { get; }  IReadOnlySet<string> Permissions { get; }
}
public sealed class ScreenDefValidator(IQuroDefinitionCatalog catalog)
{
    public IReadOnlyList<DefinitionError> Validate(ScreenDef def);
}

// ✅ D-4 — 식 (LockWhen·RuleDef.Expr는 모델에 문자열로 두고 여기서 파싱). 파싱 실패 = FormatException
// Load는 calc·sum·set 식과 lockWhen을 파싱해 문법 오류·없는 필드를 V-18, 다른 그리드 열 입력을 V-15로 보고
public sealed class RuleExpression
{
    public static RuleExpression Parse(string expr);
    public decimal? Evaluate(Func<string, decimal?> valueOf, Func<string, IEnumerable<decimal?>>? columnValues = null);   // columnValues: sum(열)용 (초안에 없던 선택 매개변수)
}
public sealed class LockExpression { public static LockExpression Parse(string text); public bool Evaluate(Func<string, object?> valueOf); }   // and가 or보다 먼저, 값은 서수 문자열 비교
```

- `SourceInfo(File, Line)`를 모든 정의에 달아 오류·로그에 위치를 낸다
- 오류는 모두 모아서 줄 순서로 돌려준다 (첫 오류에서 멈추지 않음). 오류가 하나라도 있으면 `Value`는 null
- 컬렉션은 `IReadOnlyList`로 노출하고 로드 뒤 바꾸지 않는다 (앱이 Singleton으로 캐시, R6)
- 식·조건은 모델에 원문 문자열로 둔다 — 파싱 실패가 모델 로드를 막지 않게 하고, 식 언어를 따로 버전 관리하기 위함
- TOON 리더(`ToonReader`)는 `internal`. i18n·`lookup.toon`도 읽어야 하면 그때 공개한다 (비파괴 추가)
- 업무 용어(`customer`, `soNo`)는 코드에 없다. 골든 파일에만 있다

---

10. AS IS 결함(TOBE 1.7) 대응 위치

ID	결함	설계 대응	맡는 곳	검증
R1	SQL Injection	TOON에 SQL 금지(V-30), 파라미터 쿼리	QuroGrid(메타) / QuroERP(API)	GS-D06, H7
R2	클라이언트 금액 저장	그리드 계산은 표시용, 서버 재계산	QuroERP	H8 (QuroERP)
R3	할인 부호	식은 메타에 선언, 업무가 결정 (T-10)	업무 결정	GS-D10 골든
R4	위치 결합	id 바인딩, 인덱스 API 없음	QuroGrid	GS-D01, H2
R5	빈 열에 오기록	`lookup` 대상은 열 id 목록, 존재 검사(V-15)	QuroGrid	GS-D05
R6	seq 경쟁	추가 행 `rowKey`=null, 서버 채번	QuroERP	—
R7	오류 처리 NPE	ProblemDetails	QuroERP	—

ID	결함	설계 대응	맡는 곳	검증
R8	마스터 자동 변경	별도 명령 (TOBE D5)	QuroERP	—
R9	Oracle 종속	`@today` 토큰	QuroERP	GS-D04 (토큰 문법)
R10	하드코딩 레이블	리소스 키만 (V-21)	QuroGrid	GS-D03
R11	빈 행 NaN 합계	식·합계에서 null = 0	QuroGrid	GS-D09

---

11. 구현 순서 (결정 후)

묶음	내용	시나리오
D-1	`Quro.Grid.Definition` 프로젝트, `ToonReader` strict	GS-D01, GS-D02
D-2	모델·구조 검증 V-01~V-18, `FieldModeResolver`	GS-D03~GS-D05, GS-D07
D-3	카탈로그 검증 V-19~V-32	GS-D03, GS-D06
D-4	`RuleExpression` · `LockExpression`	GS-D08~GS-D10
D-5	그리드 연결 `<QuroGrid Definition>` (T-11) + 갭 G-01~G-04	GS-D11~GS-D15
D-6	검증 CLI `quro-deflint` (T-12, `Quro.Grid.DefLint` .NET 도구), 패키지 배포	GS-D16 ✅

각 묶음은 브랜치·PR 1개, 테스트 먼저 (CLAUDE.md 3장). 공개 API가 생기므로 묶음마다 consumer-contract.md 6장과 CHANGELOG를 고친다.